Instituto Politécnico Nacional
Escuela Superior de Cómputo

Paradigmas de Programación

Sistema experto para la gestión de almacén

Documento técnico

Integrantes: Raul Recillas Palacios
Cruz Hernandez Alejandro

Grupo: 3BM2

Docente: Dra. Ursula Samantha Morales Rodriguez

Unidad de aprendizaje: Paradigmas de Programación

Ciudad de México
21 de octubre de 2026

Índice

1. Introducción **2**

2. Planteamiento del problema **2**
2.1. Usuarios previstos 2

3. Objetivo general y objetivos específicos **2**
3.1. Objetivo general 2
3.2. Objetivos específicos 2

4. Justificación del uso de Prolog **3**

5. Descripción de la base de conocimientos **3**

6. Explicación de los hechos y reglas más importantes **3**
6.1. Validación 3
6.2. Reglas de decisión 3

7. Diagrama de arquitectura **4**

8. Descripción del funcionamiento de la interfaz **4**

9. Tecnologías utilizadas **5**

10. Procedimiento de integración y despliegue **5**
10.1. Ejecución local 5
10.2. Construcción con Docker 5
10.3. Despliegue publicado 6

11. Pruebas y resultados **6**

12. Problemas encontrados y soluciones implementadas **6**

13. Conclusiones individuales **7**

14. Referencias y declaración de uso de herramientas de IA **7**
14.1. Referencias 7
14.2. Declaración de uso de herramientas de IA 7

1. Introducción

Este documento presenta el diseño y la implementación de un sistema experto para apoyar algunas decisiones de operación de un almacén. El sistema consulta un catálogo de 20 productos y, a partir de sus atributos, propone acciones relacionadas con ubicación, reposición y movimiento.

La base de conocimiento y las reglas se encuentran en `ALMACEN.pl`. Una API escrita en Python ejecuta consultas controladas en SWI-Prolog y entrega los resultados como JSON a una interfaz web. La aplicación es una herramienta académica de demostración: sus datos son fijos y sus recomendaciones no sustituyen los procedimientos reales de seguridad e inventario.

2. Planteamiento del problema

En un almacén, el personal necesita decidir dónde ubicar los productos y cuáles requieren reposición, cuidado especial o atención prioritaria. Resolver esas decisiones manualmente puede producir criterios inconsistentes, sobre todo cuando hay que considerar al mismo tiempo existencias, rotación, fragilidad, volumen, condiciones de conservación y prioridad.

El proyecto representa un conjunto acotado de esas condiciones mediante hechos y reglas lógicas. El sistema recibe el identificador de un producto del catálogo, verifica que sus datos sean utilizables y devuelve todas las recomendaciones cuyas condiciones se satisfacen. El alcance actual no incluye persistencia, edición de existencias, autenticación ni captura libre de atributos.

2.1. Usuarios previstos

El usuario previsto es personal de almacén que revisa inventario, recibe productos o prepara pedidos y necesita consultar recomendaciones operativas para un producto registrado.

3. Objetivo general y objetivos específicos

3.1. Objetivo general

Desarrollar una aplicación web que integre una base de conocimiento en SWI-Prolog con una API y una interfaz gráfica para generar recomendaciones controladas sobre productos de un almacén.

3.2. Objetivos específicos

1. Representar productos y atributos operativos mediante hechos Prolog.
2. Definir reglas para identificar ubicaciones, necesidades de reposición y acciones de movimiento.
3. Validar registros incompletos, valores fuera de catálogo y existencias negativas.
4. Exponer consultas mediante una API HTTP que acepte identificadores dentro del rango permitido.
5. Presentar al usuario las respuestas JSON devueltas por el motor lógico.

6. Registrar escenarios de prueba para entradas válidas, inválidas y productos inexistentes.

4. Justificación del uso de Prolog

Prolog es adecuado para este prototipo porque permite expresar el conocimiento como hechos y reglas declarativas. Cada regla describe las condiciones que deben cumplirse para obtener una conclusión, sin programar un recorrido imperativo por cada producto. La unificación enlaza el identificador consultado con los argumentos del hecho y el backtracking permite encontrar soluciones adicionales.

En este sistema, predicados como `ubicacion/2`, `reposicion/2` y `movimiento/2` pueden producir más de una conclusión para un producto. `findall/3` reúne esas soluciones para que el adaptador las incluya en la respuesta JSON.

5. Descripción de la base de conocimientos

La base de conocimiento está implementada en el archivo `ALMACEN.pl`. El predicado principal de datos es `producto/8`; sus argumentos son:

1. identificador numérico;
2. nombre;
3. existencias;
4. rotación: `alta`, `media` o `baja`;
5. empaque o condición del producto: `normal` o `fragil`;
6. volumen: `estandar` o `voluminoso`;
7. ambiente: `seco` o `refrigerado`;
8. prioridad: `alta`, `media`, `baja` o, en registros inválidos, otro valor no admitido.

El archivo contiene 20 hechos, con IDs del 1 al 20. Algunos registros representan datos incompletos o inválidos para que se pueda observar la respuesta de validación. La base es estática: las consultas no modifican los hechos.

6. Explicación de los hechos y reglas más importantes

6.1. Validación

`es_invalido/2` genera una explicación cuando un registro contiene campos marcados como `sin_datos`, existencias negativas o parámetros que no pertenecen a los catálogos aceptados. La API realiza además una validación de tipo y rango para el ID antes de invocar Prolog.

6.2. Reglas de decisión

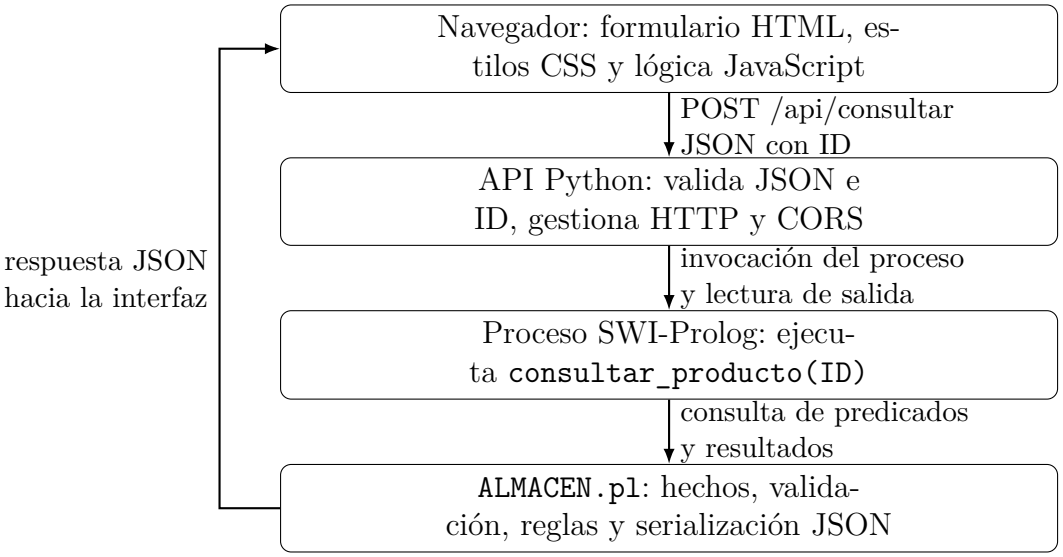
Predicado	Función y condiciones representadas
`ubicacion/2`	Recomienda zona fría para productos refrigerados, zona de voluminosos para artículos grandes, alta accesibilidad para ciertos artículos de alta rotación, zona estándar o resguardo para productos frágiles de baja rotación.

`reposicion/2`	Emite alerta crítica cuando las existencias son menores que 10; para rotación alta contempla reorden entre 10 y 30, y stock saludable por encima de 30; para rotación media o baja contempla monitoreo cuando existen al menos 10 unidades.
`movimiento/2`	Recomienda manipulación especial para productos voluminosos, empaque protector para frágiles, despacho expedito para prioridad alta, logística inversa bajo una condición de baja rotación y flujo FIFO en un caso estándar.
`obtener_json_decisiones/2`	Distingue producto inexistente, producto con errores de datos y producto válido; en el caso válido recopila ubicaciones, reposiciones y movimientos con `findall/3`.
`consultar_producto/1`	Imprime la respuesta JSON para que el proceso backend la capture por salida estándar.

Las reglas son condiciones deterministas sobre el catálogo, aunque diferentes reglas pueden cumplirse simultáneamente. Por ejemplo, un producto refrigerado, de prioridad alta y con existencias bajas puede generar recomendaciones en varias categorías. Las recomendaciones concretas dependen de los hechos y umbrales definidos en `ALMACEN.pl`.

7. Diagrama de arquitectura

La figura muestra el recorrido de una consulta desde el navegador hasta SWI-Prolog y de regreso como JSON.



El proceso externo tiene un límite de diez segundos. La API no acepta metas Prolog escritas por el usuario: construye la consulta a partir de un ID entero validado entre 1 y 20.

8. Descripción del funcionamiento de la interfaz

La interfaz presenta un campo numérico para el ID del producto, un botón de consulta, un indicador de estado y un área de resultado. El rango mostrado es de 1 a 20. Al

enviar el formulario, `app.js` evita la recarga de la página, envía el ID como JSON a `/api/consultar` y muestra la respuesta con formato legible.

Si el backend no responde, la interfaz presenta un mensaje de conexión. Si recibe una respuesta HTTP, indica si la consulta fue recibida o no fue válida y muestra el contenido devuelto. `styles.css` adapta la distribución a pantallas estrechas. La configuración de la URL remota del backend se encuentra en `frontend/config.js`.

9. Tecnologías utilizadas

Tecnología	Uso en el proyecto
SWI-Prolog	Motor lógico, hechos, reglas, consultas y biblioteca JSON.
Python 3	Servidor HTTP, validación de la solicitud y ejecución del proceso Prolog mediante `subprocess`.
HTML5	Estructura de la página y formulario.
CSS	Presentación adaptable de la interfaz.
JavaScript	Envío asíncrono de la consulta y presentación de resultados.
JSON sobre HTTP	Contrato de intercambio entre frontend y backend y entre Prolog y Python.
Docker	Construcción de una imagen que contiene SWI-Prolog, Python y la aplicación.
Vercel y Render	Direcciones de frontend y backend registradas en el documento de pruebas.

El repositorio no fija versiones exactas de Python ni de SWI-Prolog; la imagen usa la etiqueta `swipl:stable`.

10. Procedimiento de integración y despliegue

10.1. Ejecución local

Se requiere Python 3, SWI-Prolog disponible como comando `swipl` y un navegador. Desde el directorio del repositorio:

```
python3 backend/server.py
```

El servidor sirve la interfaz y queda disponible en el puerto definido por `PORT`, cuyo valor predeterminado es 8000. La comprobación de salud es http://localhost:8000/api/health.

10.2. Construcción con Docker

Desde el directorio del proyecto se puede construir y arrancar el contenedor:

```
docker build -t sistema-experto-almacen .
docker run --rm -p 8000:8000 sistema-experto-almacen
```

El `Dockerfile` parte de `swipl:stable`, instala Python 3 y copia los archivos de la aplicación.

10.3. Despliegue publicado

El registro de pruebas identifica el frontend como https://prolog-seven.vercel.app y el backend como https://prolog-d7v2.onrender.com. La variable `window.BACKEND_URL` en `frontend/config.js` apunta al backend. El backend admite configuración de puerto mediante `PORT` y de origen permitido mediante `FRONTEND_ORIGIN`. Para reproducir el despliegue, deben publicarse el frontend y el backend, configurarse esos valores de acuerdo con los dominios asignados y comprobar la ruta de salud y las consultas desde el navegador.

11. Pruebas y resultados

La tabla resume los resultados consignados en `PRUEBAS.md`; se presentan como resultados del registro del proyecto y no como una nueva ejecución realizada durante la elaboración de este documento.

ID	Escenario	Resultado registrado	Estado
P01	Existencias bajas (ID 1)	JSON válido con reposición crítica y movimiento FIFO	Aprobada
P02	Existencias suficientes (ID 2)	JSON válido con inventario saludable y monitoreo	Aprobada
P03	Alta rotación (ID 3)	Acceso y stock saludable	Aprobada
P04	Producto frágil (ID 4)	Recomendación de empaque para frágil	Aprobada
P05	Producto voluminoso (ID 5)	Zona de voluminosos y manipulación especial	Aprobada
P06	Producto refrigerado (ID 6)	Recomendación de zona fría	Aprobada
P07	Prioridad alta (ID 7)	Despacho expedito	Aprobada
P08	Datos incompletos (ID 8)	Estado JSON `invalid`	Aprobada
P09	Datos no válidos (ID 9)	Estado `invalid` con dos errores	Aprobada
P10	ID fuera de catálogo (ID 99)	HTTP 400 por ID fuera del rango permitido	Aprobada

El registro indica evidencia de respuestas locales y desplegadas para algunos casos y deja comparaciones adicionales como pendientes. También incluye una consulta para observar soluciones sucesivas mediante backtracking. La fecha, el navegador, el responsable y el hash registrado en ese documento deben revisarse y actualizarse para la entrega final.

12. Problemas encontrados y soluciones implementadas

Problema registrado	Solución implementada

El predicado `json_write_string/2` no estaba disponible en la biblioteca JSON de SWI-Prolog.	Se sustituyó por `json_write/2` y `with_output_to/2` para escribir la respuesta JSON en una cadena.

El adaptador diferencia errores de entrada y errores de ejecución del proceso, y la API define respuestas para proceso ausente, tiempo agotado y salida JSON inválida. El registro de incidencias consultado solo documenta explícitamente el problema de serialización anterior.

13. Conclusiones individuales

Raul Recillas Palacios.
Pendiente de redacción por el integrante. Completar con una conclusión personal sobre el modelado del conocimiento, el uso de Prolog y su participación en el proyecto.

Cruz Hernandez Alejandro.
Pendiente de redacción por el integrante. Completar con una conclusión personal sobre la integración de la interfaz, la API y el motor lógico, así como su participación en el proyecto.

14. Referencias y declaración de uso de herramientas de IA

14.1. Referencias

1. SWI-Prolog, documentación oficial. https://www.swi-prolog.org/pldoc/
2. Python Software Foundation, documentación de Python 3. https://docs.python.org/3/
3. MDN Web Docs, documentación de HTML, CSS, JavaScript y Fetch. https://developer.mozilla.org/
4. Docker, documentación oficial. https://docs.docker.com/
5. Vercel, documentación oficial. https://vercel.com/docs
6. Render, documentación oficial. https://render.com/docs
7. Equipo del proyecto, *Registro de pruebas*, archivo `PRUEBAS.md` del repositorio.

14.2. Declaración de uso de herramientas de IA

Se utilizaron herramientas de inteligencia artificial como apoyo para revisar el proyecto, apoyar el despliegue, embellecer el frontend y elaborar el Docker.